

4.1.3 Stack模板类

既然栈可视为序列的特例，故只要将栈作为向量的派生类，即可利用C++的继承机制，基于2.2.3节定义的向量模板类实现栈结构。当然，这里需要按照栈的习惯，对各接口重新命名。

按照表4.1所列的ADT接口，可描述并实现Stack模板类如代码4.1所示。

```
1 #include "../Vector/Vector.h" //以向量为基类，派生出栈模板类
2 template <typename T> class Stack: public Vector<T> { //将向量的首/末端作为栈底/顶
3 public: //size()、empty()以及其它开放接口，均可直接沿用
4     void push(T const& e) { insert(size(), e); } //入栈：等效于将新元素作为向量的末元素插入
5     T pop() { return remove(size() - 1); } //出栈：等效于删除向量的末元素
6     T& top() { return (*this)[size() - 1]; } //取顶：直接返回向量的末元素
7 };
```

代码4.1 Stack模板类

既然栈操作都限制于向量的末端，参与操作的元素没有任何后继，故由2.5.5节和2.5.6节的分析结论可知，以上栈接口的时间复杂度均为常数。

套用以上思路，也可直接基于3.2.2节的List模板类派生出Stack类（习题[4-1]）。

§ 4.2 栈与递归

习题[1-17]指出，递归算法所需的空间量，主要决定于最大递归深度。在达到这一深度的时刻，同时活跃的递归实例达到最多。那么，操作系统具体是如何实现函数（递归）调用的？如何记录调用与被调用函数（递归）实例之间的关系？如何实现函数（递归）调用的返回？又是如何维护同时活跃的所有函数（递归）实例的？所有这些问题的答案，都可归结于栈。

4.2.1 函数调用栈

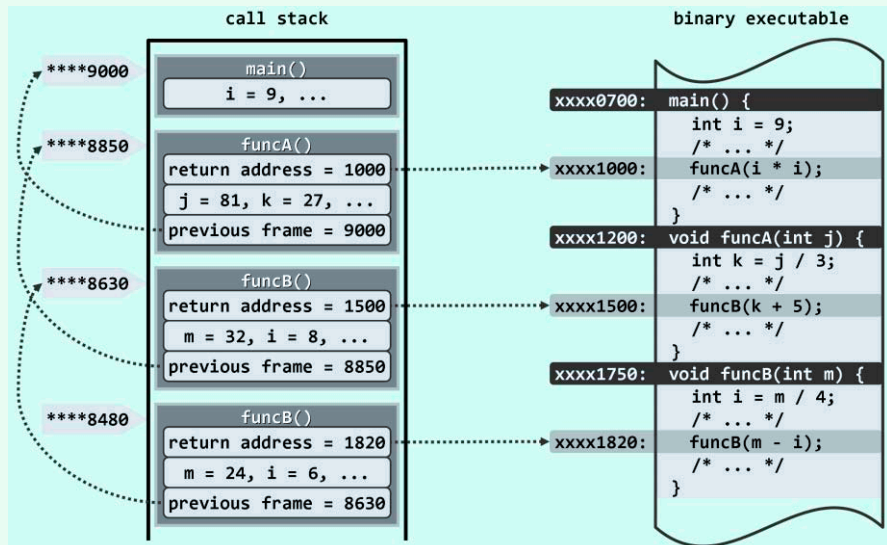


图4.3 函数调用栈实例：主函数main()调用funcA()，funcA()调用funcB()，funcB()再自我调用

在Windows等大部分操作系统中，每个运行中的二进制程序都配有一个调用栈（**call stack**）或执行栈（**execution stack**）。借助调用栈可以跟踪属于同一程序的所有函数，记录它们之间的相互调用关系，并保证在每一调用实例执行完毕之后，可以准确地返回。

■ 函数调用

如图4.3所示，调用栈的基本单位是帧（**frame**）。每次函数调用时，都会相应地创建一帧，记录该函数实例在二进制程序中的返回地址（**return address**），以及局部变量、传入参数等，并将该帧压入调用栈。若在该函数返回之前又发生新的调用，则同样地要将与新函数对应的一帧压入栈中，成为新的栈顶。函数一旦运行完毕，对应的帧随即弹出，运行控制权将被交还给该函数的上层调用函数，并按照该帧中记录的返回地址确定在二进制程序中继续执行的位置。

在任一时刻，调用栈中的各帧，依次对应于那些尚未返回的调用实例，亦即当时的活跃函数实例（**active function instance**）。特别地，位于栈底的那帧必然对应于入口主函数**main()**，若它从调用栈中弹出，则意味着整个程序的运行结束，此后控制权将交还给操作系统。

仿照递归跟踪法，程序执行过程出现过的函数实例及其调用关系，也可构成一棵树，称作该程序的运行树。任一时刻的所有活跃函数实例，在调用栈中自底到顶，对应于运行树中从根节点到最新活跃函数实例的一条调用路径。

此外，调用栈中各帧还需存放其它内容。比如，因各帧规模不一，它们还需记录前一帧的起始地址，以保证其出栈之后前一帧能正确地恢复。

■ 递归

作为函数调用的特殊形式，递归也可借助上述调用栈得以实现。比如在图4.3中，对应于**funcB()**的自我调用，也会新压入一帧。可见，同一函数可能同时拥有多个实例，并在调用栈中各自占有一帧。这些帧的结构完全相同，但其中同名的参数或变量，都是独立的副本。比如在**funcB()**的两个实例中，入口参数**m**和内部变量**i**各有一个副本。

4.2.2 避免递归

今天，包括C++在内的各种高级程序设计语言几乎都允许函数直接或间接地自我调用，通过递归来提高代码的简洁度和可读性。而Cobol和Fortran等早期的程序语言虽然一开始并未采用栈来实现过程调用，但在其最新的版本中也陆续引入了栈结构来支持过程调用。

尽管如此，系统在后台隐式地维护调用栈的过程中，难以区分哪些参数和变量是对计算过程有实质作用的，更无法以通用的方式对它们进行优化，因此不得不将描述调用现场的所有参数和变量悉数入栈。再加上每一帧都必须保存的执行返回地址以及前一帧起始位置，往往导致程序的空间效率不高甚至极低；同时，隐式的入栈和出栈操作也会令实际的运行时间增加不少。

因此在追求更高效率的场合，应尽可能地避免递归，尤其是过度的递归。实际上，我们此前已经介绍过相应的方法和技巧。例如，在1.4.4节中将尾递归转换为等效的迭代形式；在1.4.5节中采用动态规划策略，将Fibonacci数算法中的二分递归改为线性递归，直至完全消除递归。

既然递归本身就是操作系统隐式地维护一个调用栈而实现的，我们自然也可以通过显式地模拟调用栈的运转过程，实现等效的算法功能。采用这一方式，程序员可以精细地裁剪栈中各帧的内容，从而尽可能降低空间复杂度的常系数。尽管算法原递归版本的高度概括性和简洁性将大打折扣，但毕竟在空间效率方面可以获得足够的补偿。

§ 4.3 栈的典型应用

4.3.1 逆序输出

在栈所擅长解决的典型问题中，有一类具有以下共同特征：首先，虽有明确的算法，但其解答却以线性序列的形式给出；其次，无论是递归还是迭代实现，该序列都是依逆序计算输出的；最后，输入和输出规模不确定，难以事先确定盛放输出数据的容器大小。因其特有的“后进先出”特性及其在容量方面的自适应性，使用栈来解决此类问题可谓恰到好处。

■ 进制转换

考查如下问题：任给十进制整数 n ，将其转换为 λ 进制的表示形式。比如 $\lambda = 8$ 时有

$$12345_{(10)} = 30071_{(8)}$$

一般地，设 $n = (d_m \dots d_2 d_1 d_0)_{(\lambda)} = d_m \times \lambda^m + \dots + d_2 \times \lambda^2 + d_1 \times \lambda^1 + d_0 \times \lambda^0$

若记 $n_i = (d_m \dots d_{i+1} d_i)_{(\lambda)}$

则有 $d_i = n_i \% \lambda$ 和 $n_{i+1} = n_i / \lambda$

这一递推关系对应的计算流程如下。可见，其输出的确为长度不定的逆序线性序列。

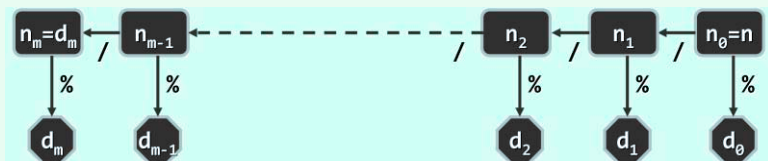


图4.4 进制转换算法流程

■ 递归实现

根据如图4.4所示的计算流程，可得到如代码4.2所示递归式算法。

```

1 void convert(Stack<char>& S, __int64 n, int base) { //十进制数n到base进制的转换（递归版）
2     static char digit[] //0 < n, 1 < base <= 16, 新进制下的数位符号，可视base取值范围适当扩充
3     = { '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F' };
4     if (0 < n) { //在尚有余数之前，不断
5         convert(S, n / base, base); //通过递归得到所有更高位
6         S.push(digit[n % base]); //输出低位
7     }
8 } //新进制下由高到低的各数位，自顶而下保存于栈S中
  
```

代码4.2 进制转换算法（递归版）

尽管新进制下的各数位须按由低到高次序逐位算出，但只要引入一个栈并将算得的数位依次入栈，则在计算结束后只需通过反复的出栈操作即可由高到低地将其顺序输出。

■ 迭代实现

这里的静态数位符号表在全局只需保留一份，但与一般的递归函数一样，该函数在递归调用栈中的每一帧都仍需记录参数 S 、 n 和 $base$ 。将它们改为全局变量固然可以节省这部分空间，但依然不能彻底地避免因调用栈操作而导致的空间和时间消耗。为此，不妨考虑改写为如代码4.3所示的迭代版本，既能充分发挥栈处理此类问题的特长，又可将空间消耗降至 $O(1)$ 。